TESTNG | CLASS 8

Excel Data-Driven Testing

Apache POI + TestNG @DataProvider + Selenium

Complete beginner-friendly notes with code explanations, real automation scenarios, reusable framework design, troubleshooting, and best practices.

Learning goal Build a reusable Excel reader that supplies many test-data rows to one TestNG test method, while keeping test data separate from test logic.

Based on the uploaded TestNG Class 8 material. The original flow and terminology are retained, with corrections and modern implementation guidance clearly explained.

Topics: Apache POI workbook/sheet/cell APIs • Object[][] • ExcelReader utility • DataProvider • supported cell types • Selenium integration • framework design • debugging and exercises

How to use these notes

Study the chapters in order. First understand how Excel is mapped into rows and columns; then learn how an ExcelReader turns that sheet into Object[][]; finally connect it to a TestNG @DataProvider and a Selenium test.

Stage	What you should be able to do
1. Understand the idea	Explain why test data and automation code should be separated.
2. Read Excel	Open an .xlsx workbook, select a sheet, inspect rows/cells and handle data types.
3. Build a reusable reader	Read a sheet dynamically, skip headers/blank rows, format values and close resources.
4. Connect TestNG	Return Object[][] from @DataProvider and map columns to test parameters.
5. Automate a scenario	Use Excel rows in a Selenium registration flow with waits, assertions and cleanup.
6. Debug / improve	Recognize common POI errors and apply maintainable framework practices.

Chapter map

1. Data-driven testing and Separation of Concerns
2. Excel structure, Apache POI objects, and row/cell indexing
3. Reading one cell and understanding data types
4. Apache POI dependencies and version compatibility
5. Reusable ExcelReader utility (complete code)
6. Understanding the utility step by step
7. TestNG @DataProvider (Object[][])
8. End-to-end Selenium example
9. Real-world testing scenarios and test-data design
10. Common errors and how to debug them
11. Best practices and framework structure
12. Hands-on exercises, solutions, interview questions and quick reference

1. What is data-driven testing?

Data-driven testing means executing the same test logic multiple times using different input data. Instead of writing a separate Java test method for every first name, phone number or city, the test receives those values from a data source such as Excel.

Simple example

FirstName	LastName	Phone	City
Amol	Jadhav	09090901001	Mumbai
Rutuja	Kale	09090901002	Pune
Sonali	Jadhav	09090901003	Bengaluru
Vikas	Mule	09090901004	Hyderabad

One registration test can run four times. Each row supplies one set of values. In practice, phone numbers and postal codes should usually be stored in Excel as text, particularly when leading zeroes are meaningful.

Without data-driven testing vs. with data-driven testing

Approach	What happens	Maintenance impact
Separate test per person	registerAmol(), registerRutuja(), registerSonali()...	Duplicate logic; fixes must be repeated.
One data-driven test	registerTestCase(firstName, lastName, phone, city)	One test implementation; add or edit rows in the data file.

Separation of Concerns (SoC)

The uploaded notes describe the Separation of Concerns principle: each layer has a distinct responsibility. Keep Excel reading, test data, test execution, page interaction and assertions from becoming one tangled block.

Component	Responsibility
Excel file	Holds input data and optional labels/notes.
ExcelReader	Loads and converts cell values into a Java data structure.
TestNG @DataProvider	Supplies each row as one test invocation.
Test method / page object	Performs UI actions and checks application behaviour.
Assertions / reports	Decide whether the observed result meets expectations and record it.

Key idea Changing a test input should not require changing the automation steps. Changing the automation steps should not require copying them into multiple test methods.

2. Excel structure and Apache POI objects

Apache POI is a Java library that can read and write Microsoft Office file formats. In the supplied class, the main focus is reading an .xlsx Excel workbook.

Excel item	Apache POI type used in the notes	Meaning
Workbook (.xlsx)	XSSFWorkbook	The Excel file and all its worksheets.
Worksheet	XSSFSheet	A named tab, for example Register.
Row	Row / XSSFRow	One horizontal record.
Cell	Cell / XSSFCell	One value at the intersection of a row and column.
Displayed cell value	DataFormatter	A formatted string suitable for many test-data fields.

Excel file formats

File extension	POI workbook type	Notes
.xlsx	XSSFWorkbook	Modern Excel XML workbook format; used by the uploaded examples.
.xls	HSSFWorkbook	Older binary Excel workbook format.
.xls or .xlsx	WorkbookFactory.create(...)	Can be useful when the reader must accept either format, provided matching POI dependencies are present.

Rows and columns are zero-based in Java

Excel displays row numbers starting at 1 and column letters starting at A. POI uses zero-based indexes: row 0 means Excel row 1; cell 0 means column A.

Excel cell	POI expression	Meaning
A1	sheet.getRow(0).getCell(0)	First row, first column.
C1	sheet.getRow(0).getCell(2)	First row, third column.
B3	sheet.getRow(2).getCell(1)	Third row, second column.
F4	sheet.getRow(3).getCell(5)	Fourth row, sixth column.

```
Row firstRow = sheet.getRow(0);
Cell firstCell = firstRow.getCell(0);
String heading = firstCell.getStringCellValue();
```

The code first gets row index 0, then cell index 0, and finally reads that cell as a String. It will throw an exception if the row/cell is missing or if the cell contains a non-string value; robust readers must check for these cases.

How to count rows and columns

```
int lastRowIndex = sheet.getLastRowNum();
int totalRowsThroughLastDefinedRow = lastRowIndex + 1;

Row firstRow = sheet.getRow(0);
int totalColumns = firstRow.getLastCellNum();
```

getLastRowNum() returns a last row index, not a row count. Adding one converts an index to a count only when counting from row zero through that index. getLastCellNum() normally returns the last cell index plus one for the row. Sparse or blank rows can make simple counts misleading.

Header reminder If row 0 contains column headings, do not pass it as a registration record. The reusable reader later in these notes intentionally starts test data at row index 1.

Worked example: one header plus three records

Excel row	POI row index	What it contains
1	0	Headers: First Name, Last Name, Phone, City
2	1	Amol, Jadhav, 9090901001, Pune
3	2	Rutuja, Kale, 9090901002, Mumbai
4	3	Sonali, Jadhav, 9090901003, Bengaluru

For this sheet, getLastRowNum() returns 3, so adding one gives 4 row positions from the header through the last defined row. If every row after the header is populated, the number of data records is 4 - 1 = 3. If blank rows exist, count nonblank records rather than relying on the index alone.

DataProvider alignment When the DataProvider returns three data rows and four columns, TestNG normally invokes the test three times, supplying four values to each invocation. The test method therefore needs four compatible parameters.

3. Read a cell and understand its data type

The source demonstrates opening the file, selecting the Register sheet and reading individual cells. A simplified example is shown below; real framework code should close both the input stream and workbook.

```
try (FileInputStream fis = new FileInputStream(filePath);
    XSSFWorkbook workbook = new XSSFWorkbook(fis)) {

   XSSFSheet sheet = workbook.getSheet("Register");
   String firstName = sheet.getRow(1).getCell(0).getStringCellValue();
   System.out.println(firstName);
}
```

The example reads cell A2 because the row index is 1 and the cell index is 0. It assumes that the sheet and cell exist and that the cell stores a text value.

Cell types that matter in a data-driven reader

CellType	Example	Recommended handling
STRING	Amol	Use getStringCellValue() or DataFormatter.
NUMERIC	1234, 2026-10-25 stored as date serial	Use DataFormatter when the formatted display is needed; use getNumericCellValue() for numeric calculations.
BOOLEAN	TRUE / FALSE	Use getBooleanCellValue() or DataFormatter.
FORMULA	=A2+B2	Use a FormulaEvaluator if you need the calculated result; getCellFormula() returns the formula text.
BLANK	Empty cell	Treat as empty string or apply an explicit validation rule.
ERROR	Excel error such as #VALUE!	Fail with a clear message rather than silently ignoring it.

Important corrections to the original class

- In Apache POI 4+ examples, use CellType.STRING and CellType.NUMERIC. CELL_TYPE_INTEGER is not a valid modern CellType enum constant; Excel numeric cells are represented by NUMERIC, not a separate integer type.
- getRawValue() is an XSSFCell-specific low-level value accessor. It is not a general-purpose way to preserve what Excel displays, and it ignores display formatting such as leading zeroes or currency formatting. Prefer DataFormatter for display-oriented test inputs.
- getCellFormula() gives a formula such as =A1+B1, not necessarily its evaluated answer. Use a FormulaEvaluator when the result is required.
- A cell with a numeric phone number can lose leading zeroes or be formatted unexpectedly. Store phone numbers, PIN codes, postal codes, IDs and account numbers as text in the Excel file unless numeric calculation is genuinely required.

Practical rule For most form fields, a reader that returns formatted cell contents as Strings is simple and reliable. Parse a value into int, long or BigDecimal only when the test actually needs numeric behaviour.

4. Apache POI Maven dependency

The uploaded lecture lists Apache POI 4.1.2 and several additional POI artifacts. Treat those versions as examples from the class, not as dependencies that must all be pasted together. Keep Apache POI artifacts on compatible, aligned versions; do not mix POI 3.9, 3.7 and 4.1.2 in one project.

```
<dependency>
   <groupId>org.apache.poi</groupId>
   <artifactId>poi-ooxml</artifactId>
   <version>4.1.2</version>
</dependency>
```

For the .xlsx/XSSFWorkbook examples in this guide, poi-ooxml normally brings the required POI core and OOXML components transitively. Use the version already chosen for your project, and verify Java compatibility before changing it. TestNG and Selenium dependencies remain separate dependencies in the automation project.

Avoid dependency duplication Do not add all of poi, poi-ooxml, poi-ooxml-schemas, ooxml-schemas, openxml4j and scratchpad at conflicting versions just because they appear in historical notes. Start with the minimum required artifact and add a specific module only when your project needs it.

5. Build a reusable ExcelReader utility

The supplied class evolves from a reader with a hard-coded local path into a reusable method: readExcelSheetValue(String fileName, String sheetName). The utility below keeps that design while adding file/sheet validation, formatted cell reads, blank-row handling, formula evaluation and safe resource closure.

```
package testutils;

import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;

import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.DataFormatter;
import org.apache.poi.ss.usermodel.FormulaEvaluator;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public final class ExcelReader {

  private ExcelReader() {
    // Utility class: do not create instances.
  }

  public static Object[][] readExcelSheetValue(
      String fileName, String sheetName) throws IOException {

    Path filePath = Paths.get(System.getProperty("user.dir"),
        "src", "test", "resources", "testdata", fileName + ".xlsx");

    if (!Files.isRegularFile(filePath)) {
      throw new IOException("Excel file not found: " + filePath);
    }

    try (InputStream input = Files.newInputStream(filePath);
       XSSFWorkbook workbook = new XSSFWorkbook(input)) {

      XSSFSheet sheet = workbook.getSheet(sheetName);
      if (sheet == null) {
        throw new IllegalArgumentException(
            "Worksheet not found: " + sheetName);
      }

      Row header = sheet.getRow(0);
      if (header == null || header.getLastCellNum() <= 0) {
        throw new IllegalArgumentException(
            "Worksheet has no header row: " + sheetName);
      }

      int totalColumns = header.getLastCellNum();
      DataFormatter formatter = new DataFormatter();
      FormulaEvaluator evaluator = workbook.getCreationHelper()
          .createFormulaEvaluator();
      List<Object[]> records = new ArrayList<>();
```

ExcelReader code, continued: read rows and return data

```
        // Start at row index 1 to skip the header row.
        for (int rowIndex = 1; rowIndex <= sheet.getLastRowNum(); rowIndex++) {
            Row row = sheet.getRow(rowIndex);
            if (row == null) {
                continue;
            }

            Object[] record = new Object[totalColumns];
            boolean allCellsBlank = true;

            for (int columnIndex = 0; columnIndex < totalColumns; columnIndex++) {
                Cell cell = row.getCell(columnIndex,
                        Row.MissingCellPolicy.RETURN_BLANK_AS_NULL);
                String value = cell == null ? ""
                        : formatter.formatCellValue(cell, evaluator);
                record[columnIndex] = value;

                if (!value.trim().isEmpty()) {
                    allCellsBlank = false;
                }
            }

            // Do not create test invocations from fully blank rows.
            if (!allCellsBlank) {
                records.add(record);
            }
        }

        if (records.isEmpty()) {
            throw new IllegalArgumentException(
                    "No test-data rows found in sheet: " + sheetName);
        }

        return records.toArray(new Object[0][]);
    }
  }
}
```

What this utility improves

Feature	Why it matters
File path from project root	Avoids machine-specific paths such as C:\Users\...; works better on teammates' machines and CI agents.
Checks file and worksheet	Fails early with a useful error instead of a confusing NullPointerException.
Header and blank-row handling	Skips column headings and avoids test invocations from fully blank rows.
DataFormatter + evaluator	Returns formatted values as strings and evaluates formulas when needed.
try-with-resources	Closes the input stream and workbook even if an exception occurs.
List<Object[]>	Allows the real number of nonblank records to determine the returned Object[][].

6. Understand ExcelReader step by step

1. Build the path. Paths.get(user.dir, ...) creates a path relative to the project root. The expected file location is src/test/resources/testdata/TestData1.xlsx.
2. Validate the file. Files.isRegularFile prevents a less helpful file-open error when the test data file does not exist at the expected path.
3. Open the stream and workbook with try-with-resources. Java closes both resources at the end of the block.
4. Select the named worksheet. getSheet(sheetName) returns null when the worksheet name is wrong, so validate it explicitly.
5. Read the header row. Its last cell number is used as the number of data columns. This implementation assumes the header defines the expected table width.
6. Create a DataFormatter and FormulaEvaluator. The formatter converts Excel values to display strings; the evaluator supplies formula results to the formatter.
7. Loop over rows starting at index 1. Index 0 is the header. A missing row is skipped.
8. Loop across the expected columns. Missing/blank cells become an empty string. Nonblank cells set allCellsBlank to false.
9. Skip fully blank rows. This prevents accidental empty TestNG invocations if there are blank lines in the worksheet.
10. Return records.toArray(new Object[0][]). TestNG accepts the resulting Object[][] as a DataProvider result.

Design decision This reader returns all cell values as Strings because registration form values are textual inputs. If a test needs numeric calculation, parse the relevant value explicitly in the test or a domain-specific data-mapping layer.

7. Excel file layout and Object[][]

Put the workbook in src/test/resources/testdata/TestData1.xlsx and name the worksheet Register. Use the first row for headings, with the remaining rows as test data.

Excel row	A: FirstName	B: LastName	C: Phone	D: City
1 (header)	FirstName	LastName	Phone	City
2	Amol	Jadhav	09090901001	Mumbai
3	Rutuja	Kale	09090901002	Pune
4	Sonali	Jadhav	09090901003	Bengaluru
5	Vikas	Mule	09090901004	Hyderabad

What Object[][] means

Object[][] is a two-dimensional Java array. Think of it as a table: the outer array holds one record per test invocation; the values inside each row are the columns supplied as method arguments.

```
Object[][] data = {
   {"Amol",   "Jadhav", "09090901001", "Mumbai"},
   {"Rutuja", "Kale",   "09090901002", "Pune"},
   {"Sonali", "Jadhav", "09090901003", "Bengaluru"}
};
```

Data row	Parameter 1	Parameter 2	Parameter 3	Parameter 4
0	Amol	Jadhav	09090901001	Mumbai
1	Rutuja	Kale	09090901002	Pune
2	Sonali	Jadhav	09090901003	Bengaluru

Three data rows mean three invocations of the same @Test method. Four columns mean four method parameters are required, in the same order. The column count does not mean four test methods.

Must match exactly If the DataProvider returns four values per row, the test method normally needs four compatible parameters. A mismatch produces a TestNG parameter-count/type error before or during invocation.

8. Connect ExcelReader to TestNG @DataProvider

The DataProvider bridges the Excel utility and the test method. It should stay small: fetch the data and return it. Do not put the whole Excel parsing loop inside each test method.

```
package tutorial3;

import java.io.IOException;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import testutils.ExcelReader;

public class RegisterDataDrivenTest {

   @DataProvider(name = "registerTestData")
   public Object[][] getTestData() throws IOException {
      return ExcelReader.readExcelSheetValue("TestData1", "Register");
   }

   @Test(dataProvider = "registerTestData")
   public void registerTestCase(String firstName, String lastName,
                                String phone, String city) {
      System.out.println("Registering: " + firstName + " " + lastName);
      System.out.println("Phone: " + phone + ", city: " + city);
      // Put the Selenium actions and assertions here.
   }
}
```

Line-by-line explanation

Code element	Purpose
@DataProvider(name = "registerTestData")	Registers a provider under a name TestNG can reference.
Object[][] getTestData()	Returns one array row per test run and one column per test argument.
ExcelReader.readExcelSheetValue(...)	Calls the reusable reader with the workbook name and sheet name.
@Test(dataProvider = "registerTestData")	Tells TestNG to invoke this test for each row returned by the provider.
String firstName, ... city	Arguments are mapped from the corresponding four columns in order.

The DataProvider method declares throws IOException because the Excel reader may fail to open/read the file. In a production framework, that failure should fail setup/data loading clearly rather than returning null or a partially filled array.

9. End-to-end Selenium example

The uploaded class fills the Guru99 New Tours registration form using Excel rows. The example below follows the same workflow but replaces fixed Thread.sleep calls with explicit waits and uses finally to close the browser. Confirm that the demo application and locators are still valid before using it; for company work, point the test at an approved test environment.

```
package tutorial3;

import java.io.IOException;
import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import testutils.ExcelReader;

public class Guru99Demo1 {

  @DataProvider(name = "registerTestData")
  public Object[][] getTestData() throws IOException {
    return ExcelReader.readExcelSheetValue("TestData1", "Register");
  }

  @Test(dataProvider = "registerTestData")
  public void registerTestCase(String firstName, String lastName,
                               String phone, String city) {
    WebDriver driver = new ChromeDriver();
    try {
      driver.manage().window().maximize();
      driver.get("https://demo.guru99.com/test/newtours/register.php");
      WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

      WebElement firstNameBox = wait.until(
          ExpectedConditions.visibilityOfElementLocated(By.name("firstName")));
      firstNameBox.sendKeys(firstName);
      driver.findElement(By.name("lastName")).sendKeys(lastName);
      driver.findElement(By.name("phone")).sendKeys(phone);
      driver.findElement(By.name("city")).sendKeys(city);

      // Verify at least that the supplied value reached the field.
      Assert.assertEquals(firstNameBox.getAttribute("value"), firstName);

      // Add the real submit action and success-message assertion
      // using current application locators.
    } finally {
      driver.quit();
    }
  }
}
```

How one invocation works

1. TestNG calls getTestData() before the test method needs its data.
2. ExcelReader loads the Register sheet and produces Object[][].
3. TestNG takes the first row and maps its four values to firstName, lastName, phone and city.
4. The test opens a browser, waits for the field, types the values and asserts that the first name was entered.
5. The finally block quits the browser whether the test passes or fails.
6. TestNG repeats the same test method for the next Excel row, then the next, until all rows have been used.

Assertion scope The sample assertion checks that text was entered into the first-name field. It does not prove that registration succeeded. For a full end-to-end test, submit the form and assert a real success message, confirmation page or persisted record.

10. Real-world scenarios

Scenario A: Positive registration tests

Use rows with valid first name, last name, phone and city. Each row should represent a unique user or a test environment that cleans up created records. Assert that the application accepts the data and displays the correct confirmation.

Scenario B: Negative validation tests

Add a column such as ExpectedResult or ScenarioType. Include invalid phone formats, missing mandatory data or a city value beyond a limit. The test must read the expected result and assert that the correct validation message appears; it should not treat every row as a successful registration.

Scenario	FirstName	Phone	ExpectedResult
Valid phone	Amol	09090901001	Registration accepted
Missing phone	Rutuja		Phone is required
Invalid phone	Sonali	ABC123	Phone format error
Boundary input	Vikas	1234567890	Apply requirement-specific expected result

Scenario C: Login or API tests

The same reader pattern can feed username/password pairs into a login test, or customer IDs/request bodies into API tests. Keep secrets out of shared workbooks when possible. Use a secure secret store/environment configuration for credentials, not a committed Excel file.

Scenario D: Multiple sheets

A workbook can contain Login, Register and Search sheets. Pass the desired sheet name to ExcelReader.readExcelSheetValue(fileName, sheetName). Keep each sheet's column layout consistent and document the expected columns.

Scenario E: Data-driven regression runs in CI

In a Maven/Jenkins pipeline, keep the workbook under src/test/resources/testdata, make sure it is committed or securely provisioned, and use a stable relative project path. A hard-coded path from one developer's Desktop will fail on another machine or build agent.

11. Common errors and debugging

Symptom / exception	Likely cause	How to fix it
FileNotFoundException / NoSuchFileException	Wrong or machine-specific file path.	Use a project-relative resources path and confirm the file name/extension.
Worksheet not found / NullPointerException	Wrong sheet name or sheet is absent.	Check exact spelling and validate sheet != null.
Cannot get a STRING value from a NUMERIC cell	Calling getStringCellValue() on a numeric cell.	Use DataFormatter or branch on cell type.
CELL_TYPE_INTEGER does not compile	Legacy or incorrect type constant.	Use CellType.NUMERIC in modern Apache POI.
ArrayIndexOutOfBounds / parameter mismatch	Object[][] columns do not match test method parameters.	Count columns and method arguments; ensure header is skipped.
Phone loses leading zeros	Excel stored the phone as a number.	Format the column as Text and use DataFormatter for display values.
Empty rows produce tests with blanks	The reader counts rows without checking content.	Skip fully blank rows or fail on malformed records.
Formula text appears instead of answer	Code uses getCellFormula().	Use FormulaEvaluator with DataFormatter if you want the calculated value.
Workbook/file lock or resources accumulate	Workbook/InputStream not closed.	Use try-with-resources.
A test passes without validating success	Script only types data or sleeps.	Assert a meaningful application outcome and avoid arbitrary sleeps.

Recommended troubleshooting sequence

1. Open the workbook manually and confirm the exact filename, sheet name, headers and data rows.
2. Print the computed absolute file path while debugging, but do not log sensitive input values.
3. Check getLastRowNum(), header.getLastCellNum() and the number of returned DataProvider rows.
4. Inspect one row and confirm the number/order of values matches the test method parameters.
5. Check Excel cell formats for phone, postal code, date and formula columns.
6. Run one test row first. After it passes, add more rows and run the full dataset.

12. Best practices for industry-quality frameworks

- Separate concerns: ExcelReader reads data; DataProvider wires data into TestNG; test/page objects perform actions; assertions validate outcomes.
- Keep test files in src/test/resources/testdata and construct paths from the project root instead of a personal Desktop path.
- Use a single, well-defined Apache POI version set. Do not mix historical versions from separate examples.

- Prefer DataFormatter for form inputs that should behave like strings. Keep phone numbers, IDs and postal codes as text in Excel.
- Skip or explicitly reject blank rows; do not silently supply null or default data for required fields.
- Validate the workbook, worksheet and header shape before executing browser tests.
- Close workbook and streams with try-with-resources. Close each WebDriver in a finally block or a lifecycle method.
- Replace Thread.sleep with WebDriverWait and ExpectedConditions.
- Write assertions that check observable business outcomes, not just that data was typed.
- Use unique data or cleanup for create operations so repeat runs do not fail because the records already exist.
- Keep credentials and production personal data out of source control and ordinary Excel test data.
- If using parallel DataProvider execution, isolate browser sessions and test data to avoid shared-state conflicts.
- Keep dataset rows traceable: include a TestCaseId, scenario type, or expected result when it improves reporting.
- Fail loudly with useful error messages if the workbook structure is invalid; avoid returning null or only printing “Invalid datatype”.

Recommended folder structure

```
project-root/
  pom.xml
  src/
    test/
      java/
        testutils/
          ExcelReader.java
        tests/
          RegisterDataDrivenTest.java
        pages/
          RegistrationPage.java      (optional Page Object Model)
      resources/
        testdata/
          TestData1.xlsx
        testng.xml                   (optional suite configuration)
```

For larger automation projects, move Selenium interactions into Page Objects and leave the TestNG class responsible for arranging data, calling page actions and asserting results. Keep Excel reading independent from Selenium so it can be reused for API tests too.

13. Hands-on exercises

Exercise 1 - explain the indexes

Excel value is in cell D5. What are the row and cell indexes used by POI?

Suggested answer: row index 4 and cell index 3, so sheet.getRow(4).getCell(3).

Exercise 2 - calculate invocations

A worksheet has one header row followed by 12 nonblank data rows. If the reader skips the header and returns all 12 rows, how many times does the DataProvider-backed @Test method run?

Suggested answer: 12 invocations, assuming every returned row is valid and no filtering removes a row.

Exercise 3 - fix a type issue

A phone cell is numeric and getStringCellValue() fails. What is the better approach for a phone field?

Suggested answer: store the phone as Text in Excel and read its displayed value with DataFormatter. Use getNumericCellValue() only if numeric calculations are required.

Exercise 4 - parameter mismatch

The DataProvider returns 4 values, but the test method declares 3 parameters. What should be changed?

Suggested answer: either return only the three intended columns or define four compatible test parameters. Ensure the order matches the Excel columns.

Exercise 5 - extend the framework

Add an ExpectedResult column to the workbook and include one invalid phone row. Update the test to assert the expected validation message instead of always expecting success.

14. Interview questions and short answers

Q. What is data-driven testing?

A. Executing the same test logic with multiple sets of input data.

Q. What is Apache POI?

A. A Java library for reading/writing Microsoft Office file formats, including Excel.

Q. What is the difference between XSSFWorkbook and HSSFWorkbook?

A. XSSFWorkbook is used for .xlsx files; HSSFWorkbook is for the older .xls format.

Q. What does getLastRowNum() return?

A. The index of the last defined row, not the number of rows.

Q. Why add 1 to getLastRowNum()?

A. To count indexes starting at zero through the last row; however, blank/sparse rows still require care.

Q. What is Object[][] in TestNG?

A. A two-dimensional array where each inner row represents one data-provider invocation.

Q. Why use @DataProvider?

A. It supplies data rows to one @Test method, causing a separate invocation for each row.

Q. What does DataFormatter do?

A. Returns a String formatted similarly to the displayed Excel cell value.

Q. Why should a phone number often be stored as text?

A. It is an identifier, not a quantity; leading zeroes and formatting should be preserved.

Q. What is Separation of Concerns?

A. A design principle that gives each component one clear responsibility.

Q. What is wrong with hard-coded Desktop paths?

A. They tie the project to one machine and break on teammates' machines or CI agents.

Q. Why use try-with-resources?

A. It reliably closes streams and workbook resources on success or failure.

Q. What is the issue with CELL_TYPE_INTEGER in modern POI?

A. Use CellType.NUMERIC; Excel cell types do not include a separate integer type in modern POI.

Q. What is the difference between getCellFormula() and formula evaluation?

A. The first returns formula text; FormulaEvaluator can calculate the formula result.

Q. What happens when data rows have four columns?

A. The test method normally needs four corresponding parameters in matching order.

Q. Should every DataProvider row represent a positive test?

A. No. Include negative and boundary scenarios with expected results, and assert each outcome correctly.

15. Quick-reference sheet

Need	API / concept	Reminder
Open .xlsx	new XSSFWorkbook(inputStream)	Use try-with-resources.
Select worksheet	workbook.getSheet("Register")	Validate the returned sheet.
Get row / cell	sheet.getRow(r).getCell(c)	Indexes are zero-based; row/cell can be null.
Count row index range	sheet.getLastRowNum() + 1	Do not mistake last index for count; consider header/blank rows.
Count cells in header	header.getLastCellNum()	Returns last cell index + 1; can be -1 for empty rows.
Format a value	new DataFormatter().formatCellValue(cell, evaluator)	Useful for text fields and formulas.
Read formula result	formatCellValue(cell, formulaEvaluator)	Do not confuse formula text with calculated output.
Provide data	@DataProvider + Object[][]	Each row maps to one invocation.
Call reader	ExcelReader.readExcelSheetValue("TestData1", "Register")	Keep file name, sheet name and schema explicit.
Close browser	finally { driver.quit(); }	Cleanup should run even if assertion fails.

Final checklist before running the suite

☐ TestData1.xlsx is present under src/test/resources/testdata.
☐ The worksheet name is exactly Register.
☐ Row 0 contains the intended headers and data starts from row 1.
☐ Every nonblank data row contains the expected columns.
☐ Phone/postal-code fields are stored and read in a way that preserves meaningful formatting.
☐ Apache POI dependencies are aligned and compatible with the project.
☐ The DataProvider name matches the dataProvider attribute exactly.
☐ Test method parameters match the number, order and type of returned columns.
☐ The Selenium test uses waits, meaningful assertions and guaranteed cleanup.
☐ Repeated test runs are safe: data is unique or the application state is cleaned up.

Remember Excel stores the inputs; ExcelReader translates them; TestNG provides them; Selenium or API code exercises the application; assertions decide whether the behaviour is correct.